

With team:

- Rank three qualities it needs most
- Write one testable goal for top quality
- Draft ADR1 for biggest structure choice
- Ask AI to critique it and log exchange

Three qualities for P1:

1. Learn applicable cybersecurity concepts
2. Provide clear feedback on user progress
3. Remove friction in tutorials, and teach user quickly

Testable goal:

- Implement the Telex machine and allow for a puzzle solving mechanic for game progression

ADR1:

Status: Proposed

Context: Team of 3, designed over 2 sprints, designed for student users to learn cybersecurity concepts through game application

Decision: Guiding a boat headed for Dunkirk beach, they will receive messages via Telex machine in order to guide them on the correct path towards the beach. It's a stretch to say a civilian would have a Telex machine, so maybe there are underwater cables that are long enough to connect to the boat. To learn applicable cybersecurity concepts, they will receive messages and have to decipher their messages in order to decide which way to steer the ship.

Consequences: There is no clear "incorrect" answer for the puzzles. If the player fails to decipher a code, then there is no indication that it was failed. Wrong answers don't delay any time, but instead don't tell the player that they're now headed on the incorrect path. Since the Telex would have to be receive-only, that also affects our ability to process correct/incorrect answers.

AI Model Response:

I read through the helm prototype chat for context. Here's where the ADR holds up and where it doesn't.

The central flaw: the Telex isn't the answer channel

The consequences section treats receive-only as the reason you can't grade the player:

Since the Telex would have to be receive-only, that also affects our ability to process correct/incorrect answers.

That doesn't follow. The player doesn't answer by typing back — they answer by **steering**. Your prototype already tracks **heading** as a live number in degrees, plus distance run on the log. If a decrypted message says "STEER 148," then the player's heading either sits within tolerance of 148 or it doesn't. That's a fully gradable answer submission, and no return path is needed.

The ADR has conflated *message delivery* with *answer input*. Once you separate them, the "no clear incorrect answer" problem mostly dissolves, and the section that currently reads as an accepted limitation should instead read as a solved design question.

The undersea cable idea will cost you more than it buys

A cable to a moving small craft isn't plausible enough to survive a student noticing it, and the ADR already sounds unconvinced of itself. Two stronger options, both historically defensible:

- **A loaned naval W/T set.** The Admiralty did put ratings and equipment aboard some of the little ships. The Ministry handing over a receiver at Ramsgate is a two-sentence justification and needs no invention.
- **An Aldis signal lamp from an escorting destroyer.** Receive-only by nature, period-correct, and flashed Morse is itself an encoding lesson.

Either keeps the receive-only constraint you want without the hand-wave. I'd note this in the ADR as alternatives considered rather than leaving "maybe there are underwater cables" in the decision text.

The vulnerability you're ignoring is the actual lesson

Right now the ciphers are Caesar-and-friends puzzles bolted to a steering sim. That's crypto history, not cybersecurity. But an unauthenticated one-way channel is *precisely* a cybersecurity concept, and the setting hands it to you:

If the Germans can transmit on the same frequency, some messages are spoofed. The player has to verify before obeying — wrong callsign, a checksum that doesn't add up, a reused key, a sequence number out of order, a message that arrives too fast to be genuine. Now you're teaching authentication, integrity, replay attacks, and key management, and the "trust nothing you can't verify" lesson lands because obeying a forged order sails you into a sandbank.

This also fixes the pedagogy gap. Deciphering a Caesar shift teaches a party trick; deciding whether a correctly-deciphered message should be trusted teaches the discipline.

Feedback without a "WRONG" popup

Your instinct that failure shouldn't produce a red buzzer is good — it just needs to become diegetic rather than absent. Silent failure with no consequence isn't restraint, it's the absence of a learning loop, and students will disengage once they realize nothing they do matters.

The prototype already gives you the instruments for this: compass, log, and landmarks placed at true bearings. Off-course feedback can be the lighthouse showing up on the wrong bow, the channel buoys passing on the wrong side, the shore staying stubbornly empty. Drift is recoverable for a while, then it costs something. The player diagnoses it themselves, which is the same skill as noticing an anomaly in a log file.

Structural problems with the ADR as a document

- **The decision isn't stated as one.** "Maybe there are underwater cables" is an open question living in the wrong section. A decision reads "We will X," with the uncertainty moved to context or a separate open-questions list.
- **No alternatives considered.** This is most of what an ADR is for. Six months from now nobody can tell whether you rejected radio, or never thought of it.
- **Context describes team logistics, not forces.** The constraints that actually bind you are missing: static HTML/CSS/JS with no backend, no persistence, three people, two sprints, unknown student baseline, and whichever curriculum concepts you're required to cover.
- **Consequences should list the good with the bad.** Currently it's three sentences of unresolved problem.

Two dependencies the ADR doesn't acknowledge

There is no position model. The prototype has heading and distance run — rotation, essentially. "Correct path" isn't representable yet. You need x/y position, waypoints, and a tolerance corridor before any of this is buildable, and that's a real chunk of one of your two sprints.

There's nowhere to put the messages. The console is already full — gauge to port, breaker panel to starboard, wheel between them — and at 390px wide you're hiding labels to make it fit. A Telex printer needs real estate or a separate surface, and on mobile that's a genuine layout decision, not a detail.

Want me to draft the rewritten ADR with these folded in?